```
use client';

import { useState } from 'react';
import { MapPin, Clock, AlertTriangle, CheckCircle, ArrowRight, Star, Navigation, Zap,
ChevronRight, X, Sparkles } from 'lucide-react';

// ============ TYPES ============
type Activity = {
  id: string;
  name: string;
  type: 'restaurant' | 'museum' | 'event' | 'transport';
  address: string;
  time: string;
  duration: string;
  price: number;
  emoji: string;
};

type Step = {
  activity: Activity;
  travel?: { mode: string; duration: string; cost: number; icon: string };
  conflict?: boolean;
};

// ============ DONNÉES MOCK (simule la BDD + API) ============
const INITIAL_ITINERARY: Step[] = [
  {
    activity: { id: '1', name: 'Petit-déjeuner au Café de Flore', type: 'restaurant', address: 'Saint-
Germain, Paris', time: '09:00', duration: '1h', price: 25, emoji: '' },
    travel: { mode: 'Métro', duration: '15 min', cost: 2.10, icon: '' }
  },
  {
    activity: { id: '2', name: 'Musée d\'Orsay', type: 'museum', address: '1 Rue de la Légion,
Paris', time: '10:30', duration: '2h30', price: 16, emoji: '' },
    travel: { mode: 'Marche', duration: '10 min', cost: 0, icon: '' }
  },
  {
    activity: { id: '3', name: 'Déjeuner chez Bouillon', type: 'restaurant', address: 'Grands
Boulevards', time: '13:15', duration: '1h15', price: 22, emoji: '' },
    travel: { mode: 'Uber', duration: '18 min', cost: 12, icon: '' },
    conflict: true //  CONFLIT SIMULÉ
  },
  {
    activity: { id: '4', name: 'Exposition Immersive', type: 'event', address: 'Grand Palais', time:
'15:00', duration: '2h', price: 18, emoji: '' }
  }
];

const SOLUTIONS = [
  { id: 'a', title: 'Décaler le déjeuner', desc: '13h45 au lieu de 13h15', type: 'shift', time: '+30
```

```
min' },
  { id: 'b', title: 'Restaurant plus proche', desc: 'Le Petit Cler — 5 min à pied', type: 'replace',
time: 'Gratuit' },
  { id: 'c', title: 'Transport plus rapide', desc: 'Vélo en libre-service', type: 'transport', time: '6
min' }
];

// ============ COMPOSANT PRINCIPAL ============
export default function Home() {
  const [itinerary, setItinerary] = useState<Step[]>(INITIAL_ITINERARY);
  const [selectedConflict, setSelectedConflict] = useState<string | null>(null);
  const [resolved, setResolved] = useState(false);

  const conflictStep = itinerary.find(s => s.conflict);
  const totalCost = itinerary.reduce((sum, s) => sum + s.activity.price + (s.travel?.cost || 0),
0);

  const applySolution = (solutionId: string) => {
    setItinerary(prev => prev.map(s => s.conflict ? { ...s, conflict: false } : s));
    setSelectedConflict(null);
    setResolved(true);
    setTimeout(() => setResolved(false), 3000);
  };

  return (
    <div className="min-h-screen bg-[#F5F5F3] text-[#171717]" style={{ fontFamily: 'Inter,
sans-serif' }}>

      {/* ============ HEADER ============ */}
      <header className="bg-white border-b border-[#E8E8E6] px-6 py-5 sticky top-0 z-20">
        <div className="max-w-2xl mx-auto flex items-center justify-between">
          <div>
            <h1 className="text-2xl font-bold" style={{ fontFamily: 'Playfair Display, serif' }}>
              VoyageZen
            </h1>
            <p className="text-sm text-[#858585]">Paris • 3 jours • 2 voyageurs</p>
          </div>
          <div className="flex items-center gap-2 bg-[#F5F5F3] px-4 py-2 rounded-full">
            <span className="text-sm font-semibold">{totalCost.toFixed(0)} €</span>
            <span className="text-xs text-[#858585]">total</span>
          </div>
        </div>
      </header>

      <main className="max-w-2xl mx-auto px-4 py-6 pb-32">

        {/* ============ BANNIÈRE RÉSOLUTION ============ */}
        {resolved && (
          <div className="mb-6 bg-green-50 border-2 border-green-500 rounded-2xl p-5 flex
items-center gap-3 animate-in fade-in slide-in-from-top-2">
```

```
          <CheckCircle className="text-green-600 w-6 h-6 flex-shrink-0" />
          <div>
            <p className="font-bold text-green-900">Itinéraire optimisé !</p>
            <p className="text-sm text-green-700">Aucun conflit détecté. Bon voyage.</p>
          </div>
        </div>
      )}

      {/* ============ TIMELINE ============ */}
      <div className="space-y-0">
        {itinerary.map((step, idx) => (
          <div key={step.activity.id}>

            {/* Carte activité */}
            <div
              onClick={() => step.conflict && setSelectedConflict(step.activity.id)}
              className={`bg-white rounded-2xl p-5 border-2 transition-all ${
                step.conflict
                  ? 'border-amber-400 shadow-md cursor-pointer hover:shadow-lg'
                  : 'border-[#E8E8E6]'
              }`}
            >
              <div className="flex items-start gap-4">
                <div className="text-3xl">{step.activity.emoji}</div>
                <div className="flex-1">
                  <div className="flex items-center gap-2 mb-1">
                    <span className="bg-[#171717] text-white text-xs font-bold px-2.5 py-1
rounded-full">
                      {step.activity.time}
                    </span>
                    {step.conflict && (
                      <span className="bg-amber-100 text-amber-800 text-xs font-bold px-2.5
py-1 rounded-full flex items-center gap-1">
                        <AlertTriangle className="w-3 h-3" />
                        Conflit
                      </span>
                    )}
                  </div>
                  <h3 className="text-lg font-bold mb-1" style={{ fontFamily: 'Playfair Display,
serif' }}>
                    {step.activity.name}
                  </h3>
                  <p className="text-sm text-[#858585] flex items-center gap-1">
                    <MapPin className="w-3 h-3" /> {step.activity.address}
                  </p>
                  <div className="flex items-center gap-3 mt-3 text-sm">
                    <span className="flex items-center gap-1"><Clock className="w-3 h-3" />
{step.activity.duration}</span>
                    <span className="font-bold">{step.activity.price} €</span>
                  </div>
```

```
              </div>
            </div>
          </div>

          {/* Connecteur transport */}
          {step.travel && idx < itinerary.length - 1 && (
            <div className="flex items-center gap-3 py-3 px-6">
              <div className="w-0.5 h-6 bg-[#E8E8E6] ml-6"></div>
              <div className="flex items-center gap-2 bg-white border border-[#E8E8E6]
rounded-full px-3 py-1.5 text-sm">
                <span>{step.travel.icon}</span>
                <span className="font-semibold">{step.travel.mode}</span>
                <span className="text-[#858585]">•</span>
                <span className="text-[#858585]">{step.travel.duration}</span>
                {step.travel.cost > 0 && (
                  <>
                    <span className="text-[#858585]">•</span>
                    <span className="font-semibold">{step.travel.cost} €</span>
                  </>
                )}
              </div>
            </div>
          )}
        </div>
      ))}
    </div>

    {/* ============ MODAL RÉSOLUTION ============ */}
    {selectedConflict && conflictStep && (
      <div className="fixed inset-0 bg-black/50 z-50 flex items-end md:items-center
justify-center p-4 animate-in fade-in">
        <div className="bg-white rounded-t-3xl md:rounded-3xl w-full max-w-lg p-6 max-
h-[85vh] overflow-y-auto">
          <div className="flex items-center justify-between mb-5">
            <div className="flex items-center gap-2">
              <div className="bg-amber-100 p-2 rounded-full">
                <Sparkles className="w-5 h-5 text-amber-700" />
              </div>
              <h2 className="text-xl font-bold" style={{ fontFamily: 'Playfair Display,
serif' }}>
                3 solutions trouvées
              </h2>
            </div>
            <button onClick={() => setSelectedConflict(null)} className="p-2 hover:bg-
[#F5F5F3] rounded-full">
              <X className="w-5 h-5" />
            </button>
          </div>

          <div className="bg-amber-50 border border-amber-200 rounded-xl p-4 mb-5">
```

```
            <p className="text-sm text-amber-900">
              Votre Uber met <strong>18 min</strong> mais vous n'avez que <strong>10 min</strong> avant l'exposition.
            </p>
          </div>

          <div className="space-y-3">
            {SOLUTIONS.map(sol => (
              <button
                key={sol.id}
                onClick={() => applySolution(sol.id)}
                className="w-full bg-[#F5F5F3] hover:bg-[#E8E8E6] border-2 border-transparent hover:border-[#171717] rounded-2xl p-4 text-left transition-all group"
              >
                <div className="flex items-center justify-between">
                  <div>
                    <p className="font-bold text-base">{sol.title}</p>
                    <p className="text-sm text-[#858585] mt-0.5">{sol.desc}</p>
                  </div>
                  <div className="flex items-center gap-2">
                    <span className="bg-white text-xs font-bold px-2 py-1 rounded-full">{sol.time}</span>
                    <ChevronRight className="w-4 h-4 text-[#858585] group-hover:text-[#171717]" />
                  </div>
                </div>
              </button>
            ))}
          </div>

          <button
            onClick={() => setSelectedConflict(null)}
            className="w-full mt-5 py-4 bg-[#171717] text-white font-bold rounded-2xl hover:bg-black transition-colors"
          >
            Garder l'itinéraire actuel
          </button>
        </div>
      </div>
    )}
  </main>

  {/* ========== BOUTON WOW ========== */}
  <div className="fixed bottom-6 left-1/2 -translate-x-1/2 z-30">
    <button className="bg-[#171717] text-white px-6 py-4 rounded-full font-bold flex items-center gap-2 shadow-2xl hover:bg-black transition-all">
      <Zap className="w-5 h-5" />
      Optimiser mon voyage
    </button>
  </div>
```

```
    </div>
  );
}
// schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id        String   @id @default(uuid())
  email     String   @unique
  name      String
  trips     Trip[]
}

model Trip {
  id          String     @id @default(uuid())
  userId      String
  user        User       @relation(fields: [userId], references: [id])
  destination String
  startDate   DateTime
  endDate     DateTime
  activities  Activity[]
}

model Activity {
  id            String          @id @default(uuid())
  tripId        String
  trip          Trip            @relation(fields: [tripId], references: [id])
  name          String
  category      String // "musee", "speakeasy", "hotel", "vol", "gratuit"
  address       String
  startTime     DateTime
  endTime       DateTime
  price         Float
  isConfirmed   Boolean         @default(false)
  transportToHere TransportOption[]
  alternatives  Alternative[]
}

model TransportOption {
  id             String   @id @default(uuid())
  activityId     String
  activity       Activity @relation(fields: [activityId], references: [id])
  mode           String // "marche", "velo", "uber", "metro", "voiture"
```

```
  durationMinutes Int
  cost            Float
  isRecommended   Boolean  @default(false)
}

model Alternative {
  id              String   @id @default(uuid())
  originalActivityId String
  originalActivity Activity @relation(fields: [originalActivityId], references: [id])
  name            String
  address         String
  startTime       DateTime
  endTime         DateTime
  price           Float
  reason          String // "Fermeture imprévue", "Conflit d'horaire"
}
// app/api/itinerary/route.ts
import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

export async function POST(req: Request) {
  const { tripId, currentActivityId, delayMinutes } = await req.json();

  // 1. Récupérer l'activité et son contexte
  const activity = await prisma.activity.findUnique({
    where: { id: currentActivityId },
    include: { transportToHere: true, alternatives: true, trip: true }
  });

  if (!activity) return NextResponse.json({ error: "Activité introuvable" }, { status: 404 });

  // 2. Simuler un imprévu (ex: retard de X minutes)
  const newStartTime = new Date(activity.startTime.getTime() + delayMinutes * 60000);
  const conflict = newStartTime >= activity.endTime;

  // 3. Générer des solutions si conflit
  let solutions = [];
  if (conflict) {
    solutions = activity.alternatives.map(alt => ({
      type: "REMPLACEMENT",
      name: alt.name,
      address: alt.address,
      newTime: alt.startTime,
      price: alt.price,
      reason: alt.reason
    }));

    // Ajouter une solution de décalage simple
```

```
    solutions.push({
      type: "DECALAGE",
      name: `Décaler "${activity.name}"`,
      newTime: new Date(activity.endTime.getTime() + 30 * 60000), // +30 min
      price: activity.price,
      reason: "Conflit d'horaire résolu par décalage"
    });
  }

  // 4. Calculer les options de transport (Mock pour l'exemple)
  const transportOptions = activity.transportToHere.map(t => ({
    mode: t.mode,
    duration: t.durationMinutes,
    cost: t.cost,
    accessible: t.mode === "marche" || t.mode === "metro" // Accessible à tous les âges
  }));

  return NextResponse.json({
    original: activity,
    newStartTime,
    hasConflict: conflict,
    transportOptions,
    solutions
  });
}
// app/trip/[id]/page.tsx
'use client';

import { useState } from 'react';

// Données mockées pour l'affichage initial
const MOCK_ITINERARY = [
  {
    id: '1',
    time: '09:00',
    name: 'Petit-déjeuner au Café de Flore',
    category: 'Restaurant',
    address: '172 Blvd Saint-Germain, Paris',
    duration: '1h',
    price: 25,
    transportNext: { mode: 'Marche', duration: '15 min', cost: 0 }
  },
  {
    id: '2',
    time: '10:30',
    name: 'Musée d\'Orsay',
    category: 'Musée',
    address: '1 Rue de la Légion d\'Honneur, Paris',
    duration: '2h',
    price: 16,
```

```
    transportNext: { mode: 'Métro', duration: '20 min', cost: 2.10 }
  }
];

export default function ItineraryPage() {
  const [selectedStep, setSelectedStep] = useState<string | null>(null);
  const [showAlternatives, setShowAlternatives] = useState(false);

  return (
    <div className="min-h-screen bg-[#F5F5F3] text-[#171717] font-sans">
      {/* En-tête simple */}
      <header className="bg-white border-b border-[#E8E8E6] p-6 sticky top-0 z-10">
        <h1 className="text-3xl font-serif font-bold text-[#171717]">Votre séjour à Paris</
h1>
        <p className="text-[#858585] text-lg mt-1">3 jours • 2 voyageurs</p>
      </header>

      <main className="max-w-4xl mx-auto p-6 grid grid-cols-1 md:grid-cols-3 gap-8">

        {/* Colonne Gauche : Chronologie */}
        <div className="md:col-span-2 space-y-6">
          {MOCK_ITINERARY.map((step, index) => (
            <div
              key={step.id}
              onClick={() => setSelectedStep(step.id)}
              className={`bg-white rounded-2xl p-6 border-2 cursor-pointer transition-all ${
                selectedStep === step.id ? 'border-black shadow-md' : 'border-[#E8E8E6]
hover:border-[#858585]'
              }`}
            >
              <div className="flex justify-between items-start">
                <div>
                  <span className="inline-block bg-[#171717] text-white text-sm font-bold px-3
py-1 rounded-full mb-3">
                    {step.time}
                  </span>
                  <h2 className="text-2xl font-serif font-semibold mb-1">{step.name}</h2>
                  <p className="text-[#858585] text-lg flex items-center gap-2">
                    {step.address}
                  </p>
                </div>
                <div className="text-right">
                  <p className="text-xl font-bold">{step.price} €</p>
                  <p className="text-[#858585]">{step.duration}</p>
                </div>
              </div>

              {/* Transport vers l'étape suivante */}
              {index < MOCK_ITINERARY.length - 1 && (
                <div className="mt-6 pt-4 border-t border-[#E8E8E6] flex items-center gap-4
```

```
text-lg">
                <span className="bg-[#F5F5F3] p-3 rounded-full"></span>
                <div>
                  <p className="font-semibold">Trajet suivant : {step.transportNext.mode}</p>
                  <p className="text-[#858585]">{step.transportNext.duration} •
{step.transportNext.cost} €</p>
                </div>
                <button className="ml-auto text-sm font-bold underline decoration-2 underline-
offset-4">
                  Changer le transport
                </button>
              </div>
            )}
          </div>
        ))}
      </div>

      {/* Colonne Droite : Carte et Actions Contextuelles */}
      <div className="space-y-6">
        {/* Placeholder Carte */}
        <div className="bg-[#E8E8E6] rounded-2xl h-64 flex items-center justify-center
border border-[#D0D0CE]">
          <p className="text-[#858585] font-semibold text-lg">[ Carte Interactive ]</p>
        </div>

        {/* Panneau d'alerte et solution (Simulé) */}
        {selectedStep === '2' && (
          <div className="bg-white rounded-2xl p-6 border-2 border-amber-400 shadow-
sm">
            <div className="flex items-center gap-3 mb-4">
              <span className="text-2xl"></span>
              <h3 className="text-xl font-bold text-amber-700">Conflit détecté</h3>
            </div>
            <p className="text-lg mb-6">
              Le Musée d'Orsay ferme à 18h00. Votre trajet en métro prend 20 min. Vous
arriverez à 17h50.
            </p>

            <div className="space-y-3">
              <button
                onClick={() => setShowAlternatives(!showAlternatives)}
                className="w-full bg-[#171717] text-white text-lg font-bold py-4 rounded-xl
hover:bg-black transition-colors"
              >
                Voir les solutions de repli
              </button>

              {showAlternatives && (
                <div className="space-y-3 mt-4 animate-in fade-in slide-in-from-top-2">
                  <div className="p-4 bg-[#F5F5F3] rounded-xl border border-[#E8E8E6]">
```

```
                  <p className="font-bold text-lg">Option 1 : Décaler le musée</p>
                  <p className="text-[#858585]">Nouvel horaire : Demain à 10h00</p>
                  <button className="mt-2 text-sm font-bold underline">Appliquer</button>
                </div>
                <div className="p-4 bg-[#F5F5F3] rounded-xl border border-[#E8E8E6]">
                  <p className="font-bold text-lg">Option 2 : Activité de remplacement</p>
                  <p className="text-[#858585]">Galerie de l'Immobilier (à 5 min à pied)</p>
                  <p className="text-sm font-semibold mt-1">Gratuit • Ouvert jusqu'à 19h00</
p>
                  <button className="mt-2 text-sm font-bold underline">Remplacer</button>
                </div>
              </div>
            )}
          </div>
        </div>
      )}
    </div>
  </main>
 </div>
 );
}
// app/api/itinerary/route.ts
import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

export async function POST(req: Request) {
 const { tripId, currentActivityId, delayMinutes } = await req.json();

 // 1. Récupérer l'activité et son contexte
 const activity = await prisma.activity.findUnique({
  where: { id: currentActivityId },
  include: { transportToHere: true, alternatives: true, trip: true }
 });

 if (!activity) return NextResponse.json({ error: "Activité introuvable" }, { status: 404 });

 // 2. Simuler un imprévu (ex: retard de X minutes)
 const newStartTime = new Date(activity.startTime.getTime() + delayMinutes * 60000);
 const conflict = newStartTime >= activity.endTime;

 // 3. Générer des solutions si conflit
 let solutions = [];
 if (conflict) {
  solutions = activity.alternatives.map(alt => ({
   type: "REMPLACEMENT",
   name: alt.name,
   address: alt.address,
   newTime: alt.startTime,
```

```
      price: alt.price,
      reason: alt.reason
    }));

    // Ajouter une solution de décalage simple
    solutions.push({
      type: "DECALAGE",
      name: `Décaler "${activity.name}"`,
      newTime: new Date(activity.endTime.getTime() + 30 * 60000), // +30 min
      price: activity.price,
      reason: "Conflit d'horaire résolu par décalage"
    });
  }

  // 4. Calculer les options de transport (Mock pour l'exemple)
  const transportOptions = activity.transportToHere.map(t => ({
    mode: t.mode,
    duration: t.durationMinutes,
    cost: t.cost,
    accessible: t.mode === "marche" || t.mode === "metro" // Accessible à tous les âges
  }));

  return NextResponse.json({
    original: activity,
    newStartTime,
    hasConflict: conflict,
    transportOptions,
    solutions
  });
}
// schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id        String   @id @default(uuid())
  email     String   @unique
  name      String
  trips     Trip[]
}

model Trip {
  id          String     @id @default(uuid())
  userId      String
```

```
  user        User       @relation(fields: [userId], references: [id])
  destination String
  startDate   DateTime
  endDate     DateTime
  activities  Activity[]
}

model Activity {
  id           String          @id @default(uuid())
  tripId       String
  trip         Trip            @relation(fields: [tripId], references: [id])
  name         String
  category     String // "musee", "speakeasy", "hotel", "vol", "gratuit"
  address      String
  startTime    DateTime
  endTime      DateTime
  price        Float
  isConfirmed  Boolean         @default(false)
  transportToHere TransportOption[]
  alternatives  Alternative[]
}

model TransportOption {
  id             String   @id @default(uuid())
  activityId     String
  activity       Activity @relation(fields: [activityId], references: [id])
  mode           String // "marche", "velo", "uber", "metro", "voiture"
  durationMinutes Int
  cost           Float
  isRecommended  Boolean  @default(false)
}

model Alternative {
  id             String   @id @default(uuid())
  originalActivityId String
  originalActivity Activity @relation(fields: [originalActivityId], references: [id])
  name           String
  address        String
  startTime      DateTime
  endTime        DateTime
  price          Float
  reason         String // "Fermeture imprévue", "Conflit d'horaire"
}
avec image photo interactif

 optimise le tous
# RÔLE ET OBJECTIF
Tu es un Ingénieur Full-Stack Senior et Expert UI/UX spécialisé en Next.js, TypeScript et design minimaliste premium.
Ta mission est de générer une application web de planification de voyage "All-in-One",
```

épurée, intuitive pour tous les âges, et techniquement optimisée.
L'application doit combiner : découverte d'activités, planificateur d'itinéraire intelligent, assistant de voyage et gestion des restrictions (visas, météo, règles locales).
Inspiration fonctionnelle : Polarsteps. Inspiration visuelle : minimalisme premium, éditorial, calme.
Langue de l'interface et des données mockées : Français naturel (ex: Paris, Rome, Lyon).

# STACK TECHNIQUE IMPOSÉE
- Framework : Next.js 15 (App Router, Server/Client Components séparés).
- Langage : TypeScript (strict).
- Styling : Tailwind CSS + shadcn/ui (pour des composants accessibles et épurés).
- Icônes : Lucide React (traits fins, minimalistes).
- État global : Zustand (léger et performant).
- Données : React Query (TanStack Query) + données mockées réalistes en local (pas de faux appels API externes pour l'instant).
- Formulaires : React Hook Form + Zod.

# DESIGN SYSTEM & ACCESSIBILITÉ (Intuitif pour tous les âges)
- Couleurs : Fond #F5F5F3, Cartes #FFFFFF, Texte principal #171717, Texte secondaire #858585, Bordures #E8E8E6.
- Accents : Noir (#000000) pour les boutons principaux. Vert/Orange/Rouge utilisés UNIQUEMENT pour les statuts et alertes.
- Typographie : Serif élégant (ex: Playfair Display ou Merriweather) pour les titres de destination. Sans-serif propre (ex: Inter ou Geist) pour l'UI, les formulaires et les données.
- Formes : Rayon de bordure généreux (16px à 24px) pour les cartes. Ombres très subtiles (`shadow-sm`).
- Accessibilité (FALC - Facile à Lire et à Comprendre) : Gros contrastes, zones de clic larges (min 44px), langage simple, pas de jargon technique, divulgation progressive des options avancées.

# ARCHITECTURE DES VUES (Priorité au MVP)
Ne génère pas tout d'un coup. Concentre-toi sur ces 4 vues principales interconnectées :
1. VUE EXPLORATION (Dashboard)
- En-tête : "Bonjour, [Prénom]", avatar, notifications.
- Module de recherche principal épuré : Adresse de départ, dates, durée, nombre de voyageurs.
- Catégories visuelles (icônes + texte) : Gastronomie, Culture, Nature, Insolite.
- Carrousels horizontaux de recommandations personnalisées (Carte avec photo, nom, lieu, durée, prix, bouton "Ajouter").
2. VUE RÉSULTATS DE RECHERCHE (Carte + Liste)
- Layout split-screen (Desktop) : Liste de cartes à gauche, carte interactive (mockée ou via composant simple) à droite.
- Filtres simplifiés (divulgation progressive) : Distance, Budget, Accessibilité (PMR), Intérieur/Extérieur.
- Chaque carte : Photo, nom, distance, temps de trajet estimé, horaires, note, bouton "Ajouter au voyage".
3. VUE PLANIFICATEUR D'ITINÉRAIRE (Cœur du produit)
- Layout 3 colonnes (Desktop) / Onglets empilés (Mobile) :
- Gauche : Timeline chronologique de la journée (ex: "Jour 1 - Rome", 09:00 Petit-déj, 10:00 Colisée). Drag-and-drop simulé ou boutons "Monter/Descendre".

- Centre : Carte avec les étapes numérotées et les trajets tracés.
- Droite : Panneau de contexte (Météo du jour, Budget cumulé, Alertes).
- Actions claires : "Optimiser les trajets", "Ajouter une étape", "Changer de jour".
4. VUE ALERTES & RESTRICTIONS (Différenciateur clé)
- Détection de conflits : Ex: "Attention : 18 min de trajet pour seulement 10 min disponibles."
- Indicateurs visuels : Badge Vert (OK), Orange (Serré), Rouge (Conflit).
- Restrictions de destination : Checklist intégrée (ex: "Passeport valide 6 mois", "Zone à faibles émissions à Rome", "Vaccination non requise"). Source et date de mise à jour affichées.
- Actions contextuelles proposées : "Décaler de 30 min", "Choisir un un autre transport", "Remplacer par une activité proche".

# DONNÉES MOCKÉES (Exemple réaliste à intégrer)
Crée un fichier `mock-data.ts` avec :
- Un utilisateur : "Evan", préférences : Gastronomie, Rythme équilibré.
- Un voyage en cours : "Rome, 3 jours", avec 5 activités planifiées le Jour 1.
- Un conflit simulé : Une activité au Panthéon prévue à 17h20, mais la fermeture est à 17h30, avec un temps de trajet depuis l'activité précédente de 18 minutes.
- Des restrictions actives pour l'Italie : "Pas de restriction vaccinale", "Taxe de séjour à payer sur place".

# CONTRAINTES DE GÉNÉRATION (RÈGLES STRICTES)
1. Commence par initialiser la structure du projet Next.js, les dépendances et le `tailwind.config.ts` avec les couleurs imposées.
2. Crée des composants UI réutilisables et atomiques (`ActivityCard`, `TimelineItem`, `AlertBanner`,
`SearchBar`).
3. Utilise des Server Components pour le rendu initial des données mockées, et des Client Components uniquement pour l'interactivité (Zustand, formulaires, carte).
4. Ne crée pas de faux appels réseau complexes ; utilise des données statiques bien structurées pour simuler l'expérience.
5. Garantir le responsive design : Mobile-first, avec une barre de navigation inférieure sur mobile et une sidebar épurée sur desktop.
6. Le code doit être propre, commenté en français, et prêt pour la production (gestion des états de chargement et des états vides).

# PREMIÈRE ACTION ATTENDUE
Génère d'abord la structure des dossiers, le `tailwind.config.ts`, les tokens de design, et le composant principal de la page "Planificateur d'itinéraire" (Vue 3) avec le conflit simulé et les données mockées, afin que je puisse valider l'identité visuelle et la logique avant de développer les autres vues.

```
// schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
```

```
  url      = env("DATABASE_URL")
}

model User {
  id        String   @id @default(uuid())
  email     String   @unique
  name      String
  trips     Trip[]
}

model Trip {
  id          String     @id @default(uuid())
  userId      String
  user        User       @relation(fields: [userId], references: [id])
  destination String
  startDate   DateTime
  endDate     DateTime
  activities  Activity[]
}

model Activity {
  id             String          @id @default(uuid())
  tripId         String
  trip           Trip            @relation(fields: [tripId], references: [id])
  name           String
  category       String // "musee", "speakeasy", "hotel", "vol", "gratuit"
  address        String
  startTime      DateTime
  endTime        DateTime
  price          Float
  isConfirmed    Boolean         @default(false)
  transportToHere TransportOption[]
  alternatives   Alternative[]
}

model TransportOption {
  id             String   @id @default(uuid())
  activityId     String
  activity       Activity @relation(fields: [activityId], references: [id])
  mode           String // "marche", "velo", "uber", "metro", "voiture"
  durationMinutes Int
  cost           Float
  isRecommended  Boolean  @default(false)
}

model Alternative {
  id                String   @id @default(uuid())
  originalActivityId String
  originalActivity  Activity @relation(fields: [originalActivityId], references: [id])
  name              String
```

```
  address           String
  startTime         DateTime
  endTime           DateTime
  price             Float
  reason            String // "Fermeture imprévue", "Conflit d'horaire"
}

// app/api/itinerary/route.ts
import { NextResponse } from 'next/server';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

export async function POST(req: Request) {
  const { tripId, currentActivityId, delayMinutes } = await req.json();

  // 1. Récupérer l'activité et son contexte
  const activity = await prisma.activity.findUnique({
    where: { id: currentActivityId },
    include: { transportToHere: true, alternatives: true, trip: true }
  });

  if (!activity) return NextResponse.json({ error: "Activité introuvable" }, { status: 404 });

  // 2. Simuler un imprévu (ex: retard de X minutes)
  const newStartTime = new Date(activity.startTime.getTime() + delayMinutes * 60000);
  const conflict = newStartTime >= activity.endTime;

  // 3. Générer des solutions si conflit
  let solutions = [];
  if (conflict) {
    solutions = activity.alternatives.map(alt => ({
      type: "REMPLACEMENT",
      name: alt.name,
      address: alt.address,
      newTime: alt.startTime,
      price: alt.price,
      reason: alt.reason
    }));

    // Ajouter une solution de décalage simple
    solutions.push({
      type: "DECALAGE",
      name: `Décaler "${activity.name}"`,
      newTime: new Date(activity.endTime.getTime() + 30 * 60000), // +30 min
      price: activity.price,
      reason: "Conflit d'horaire résolu par décalage"
    });
  }
```

```
  // 4. Calculer les options de transport (Mock pour l'exemple)
  const transportOptions = activity.transportToHere.map(t => ({
    mode: t.mode,
    duration: t.durationMinutes,
    cost: t.cost,
    accessible: t.mode === "marche" || t.mode === "metro" // Accessible à tous les âges
  }));

  return NextResponse.json({
    original: activity,
    newStartTime,
    hasConflict: conflict,
    transportOptions,
    solutions
  });
}

// app/trip/[id]/page.tsx
'use client';
import { useState } from 'react';

// Données mockées pour l'affichage initial
const MOCK_ITINERARY = [
  {
    id: '1',
    time: '09:00',
    name: 'Petit-déjeuner au Café de Flore',
    category: 'Restaurant',
    address: '172 Blvd Saint-Germain, Paris',
    duration: '1h',
    price: 25,
    transportNext: { mode: 'Marche', duration: '15 min', cost: 0 }
  },
  {
    id: '2',
    time: '10:30',
    name: 'Musée d\'Orsay',
    category: 'Musée',
    address: '1 Rue de la Légion d\'Honneur, Paris',
    duration: '2h',
    price: 16,
    transportNext: { mode: 'Métro', duration: '20 min', cost: 2.10 }
  }
];

export default function ItineraryPage() {
  const [selectedStep, setSelectedStep] = useState<string | null>(null);
  const [showAlternatives, setShowAlternatives] = useState(false);

  return (
```

```
   <div className="min-h-screen bg-[#F5F5F3] text-[#171717] font-sans">
     {/* En-tête simple */}
     <header className="bg-white border-b border-[#E8E8E6] p-6 sticky top-0 z-10">
       <h1 className="text-3xl font-serif font-bold text-[#171717]">Votre séjour à Paris</
h1>
       <p className="text-[#858585] text-lg mt-1">3 jours • 2 voyageurs</p>
     </header>

     <main className="max-w-4xl mx-auto p-6 grid grid-cols-1 md:grid-cols-3 gap-8">
       {/* Colonne Gauche : Chronologie */}
       <div className="md:col-span-2 space-y-6">
         {MOCK_ITINERARY.map((step, index) => (
           <div
             key={step.id}
             onClick={() => setSelectedStep(step.id)}
             className={`bg-white rounded-2xl p-6 border-2 cursor-pointer transition-all ${
               selectedStep === step.id ? 'border-black shadow-md' : 'border-[#E8E8E6]
hover:border-[#858585]'
             }`}
           >
             <div className="flex justify-between items-start">
               <div>
                 <span className="inline-block bg-[#171717] text-white text-sm font-bold px-3
py-1 rounded-full mb-3">
                   {step.time}
                 </span>
                 <h2 className="text-2xl font-serif font-semibold mb-1">{step.name}</h2>
                 <p className="text-[#858585] text-lg flex items-center gap-2">
                   📍 {step.address}
                 </p>
               </div>
               <div className="text-right">
                 <p className="text-xl font-bold">{step.price} €</p>
                 <p className="text-[#858585]">{step.duration}</p>
               </div>
             </div>

             {/* Transport vers l'étape suivante */}
             {index < MOCK_ITINERARY.length - 1 && (
               <div className="mt-6 pt-4 border-t border-[#E8E8E6] flex items-center gap-4
text-lg">
                 <span className="bg-[#F5F5F3] p-3 rounded-full">🚶</span>
                 <div>
                   <p className="font-semibold">Trajet suivant : {step.transportNext.mode}</p>
                   <p className="text-[#858585]">{step.transportNext.duration} •
{step.transportNext.cost} €</p>
                 </div>
                 <button className="ml-auto text-sm font-bold underline decoration-2 underline-
offset-4">
                   Changer le transport
```

```
              </button>
            </div>
          )}
        </div>
      ))}
    </div>

    {/* Colonne Droite : Carte et Actions Contextuelles */}
    <div className="space-y-6">
      {/* Placeholder Carte */}
      <div className="bg-[#E8E8E6] rounded-2xl h-64 flex items-center justify-center
border border-[#D0D0CE]">
        <p className="text-[#858585] font-semibold text-lg">[ Carte Interactive ]</p>
      </div>

      {/* Panneau d'alerte et solution (Simulé) */}
      {selectedStep === '2' && (
        <div className="bg-white rounded-2xl p-6 border-2 border-amber-400 shadow-
sm">
          <div className="flex items-center gap-3 mb-4">
            <span className="text-2xl">⚠️</span>
            <h3 className="text-xl font-bold text-amber-700">Conflit détecté</h3>
          </div>
          <p className="text-lg mb-6">
            Le Musée d'Orsay ferme à 18h00. Votre trajet en métro prend 20 min. Vous
arriverez à 17h50.
          </p>
          <div className="space-y-3">
            <button
              onClick={() => setShowAlternatives(!showAlternatives)}
              className="w-full bg-[#171717] text-white text-lg font-bold py-4 rounded-xl
hover:bg-black transition-colors"
            >
              Voir les solutions de repli
            </button>
            {showAlternatives && (
              <div className="space-y-3 mt-4 animate-in fade-in slide-in-from-top-2">
                <div className="p-4 bg-[#F5F5F3] rounded-xl border border-[#E8E8E6]">
                  <p className="font-bold text-lg">Option 1 : Décaler le musée</p>
                  <p className="text-[#858585]">Nouvel horaire : Demain à 10h00</p>
                  <button className="mt-2 text-sm font-bold underline">Appliquer</button>
                </div>
                <div className="p-4 bg-[#F5F5F3] rounded-xl border border-[#E8E8E6]">
                  <p className="font-bold text-lg">Option 2 : Activité de remplacement</p>
                  <p className="text-[#858585]">Galerie de l'Immobilier (à 5 min à pied)</p>
                  <p className="text-sm font-semibold mt-1">Gratuit • Ouvert jusqu'à 19h00</
p>
                  <button className="mt-2 text-sm font-bold underline">Remplacer</button>
                </div>
              </div>
```

```
              )}
            </div>
          </div>
        )}
      </div>
    </main>
  </div>
 );
}
```